Технологии глубокого обучения и нейронных сетей

Пара 1: что такое языковая модель

Петр Георгиевич Меньших · МТУСИ

Как устроен день

Пара 1

Что такое языковая модель и откуда ее уверенные «ошибки»

Подводка к работе 1: модель из счетчиков и свой токенизатор

Пара 2

Как устроен трансформер и как его обучают

Подводка к работе 2: свой трансформер, обученный на книге

Две лабораторные работы

Репозиторий из шаблона курса на GitHub, автопроверка

Применение LLM в курсе про LLM

Три правила

1

Вы отвечаете за каждую строку

На защите можете объяснить и изменить свой код без ассистента.

2

Перепроверяйте модель

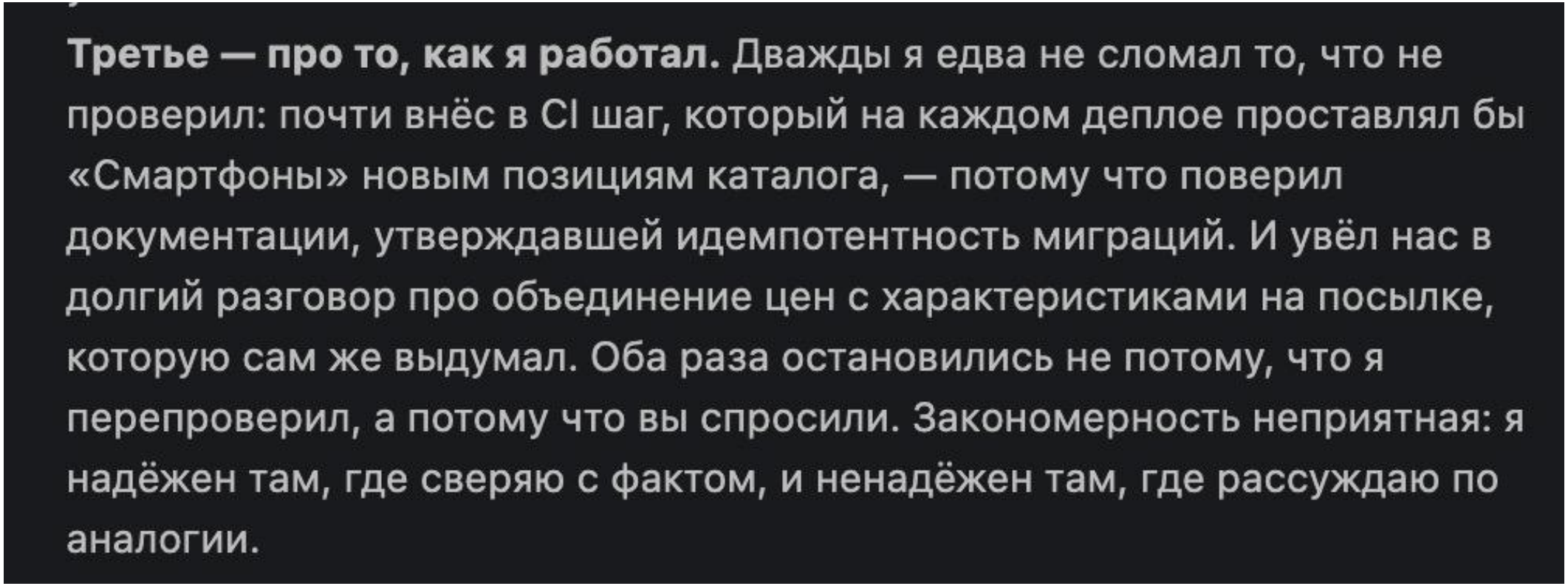
Она ошибается также уверенно, как и дает верные ответы.

3

Не теряйте свой проект

ИИ — это инструмент, а не замена.

Пример контроля с предыдущего слайда



Фронтирная модель Claude Opus 5.5

Что такое машинное обучение

Данные → модель → предсказание на новых данных

С учителем

Есть правильные ответы. Регрессия предсказывает число, классификация предсказывает класс.

Без учителя

Ответов нет, ищем структуру: кластеризация, снижение размерности.

С подкреплением

Есть среда и награда. Агент учится действовать.

Генерация текста тоже задача с учителем. Учитель это сам текст: следующее слово всегда известно.

Как модель понимает, что ошиблась

MSE

Для чисел.

$(y - \hat{y})^2$

MAE

Для чисел с выбросами.

$|y - \hat{y}|$

Кросс-энтропия

Для классов.

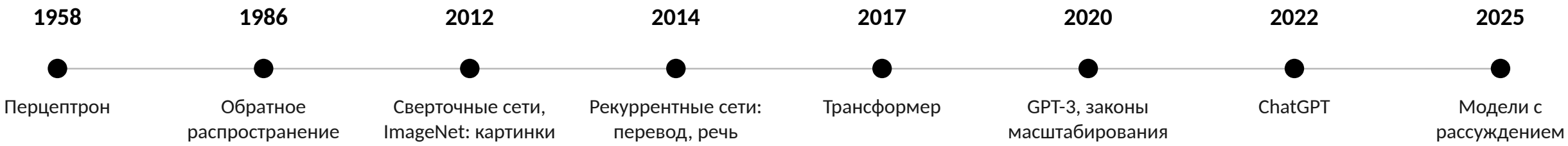
$-\log \hat{p}$

Функция потерь это число: чем больше, тем хуже.

Обучение это подбор параметров, при которых это число минимально. Больше в обучении ничего нет.

Почему именно квадрат и логарифм: заметка why-loss.md в репозитории, для любознательных.

От перцептрона до языковой модели



Пятьдесят лет идея ждала данных и вычислений. Потом за десять лет: картинки, звук, текст.

Одна архитектура закрывает почти все задачи с текстом.

Где читать

Все ссылки в README репозитория курса

- Курс Е. Соколова (ml-course-hse) и Яндекс ML Book: начинать отсюда, ближе к практике
- Курсы К. В. Воронцова и А. Г. Дьяконова: когда захочется строгости
- Deep Learning School (DLS): нейросети с нуля
- В. Бабушкин, «Machine Learning System Design»: для тех, кто строит сервисы
- Ф. Шолле, «Глубокое обучение на Python»: от тензора до своей GPT
- Kaggle, Papers with Code: практика и передний край
- Документация scikit-learn, PyTorch, transformers

github.com/pmenshih/tgo-course

Монетка

100 бросков, 63 орла.

Какова вероятность орла?

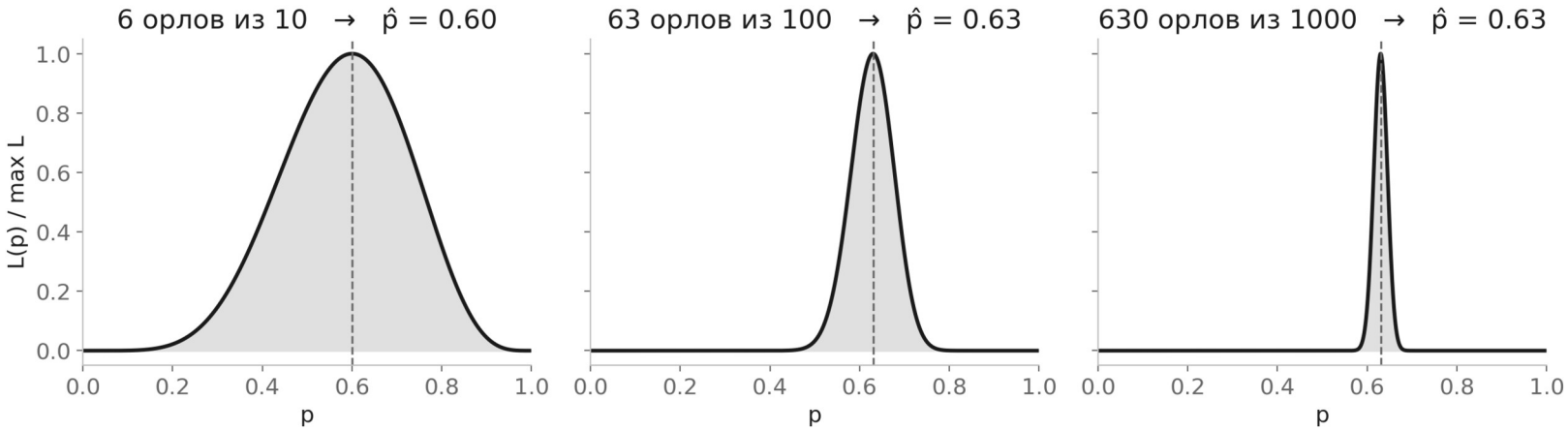
Ответ очевиден. Но откуда он взялся? Тот же вопрос стоит перед любой моделью.

0.63

Правдоподобие

$$L(p) = p^{63}\,(1-p)^{37}$$

Насколько вероятны наши данные, 63 орла и 37 решек, при таком p.



Вершина говорит, какой ответ лучший.

Ширина говорит, насколько ему верить.

Больше данных, острее пик.

ПРИНЦИП

Выбираем параметр, при котором данные наиболее правдоподобны. Это и есть обучение.

Потеря это минус логарифм вероятности

потеря = −log P(правильный ответ)

Обучение делает вероятность правильных ответов как можно больше. Функция потерь это та же вероятность: с логарифмом, чтобы произведение стало суммой, и с минусом, чтобы минимизировать.

Ошибки разбросаны колоколом

Минус логарифм дает квадрат. Это MSE.

Ответ «да или нет»

Минус логарифм дает кросс-энтропию.

Три потери со слайда про функцию потерь это одно правило при разных предположениях о данных. Вывод: why-loss.md.

Модель — семейство распределений.

Обучение — выбор члена семейства, при котором данные наиболее правдоподобны.

Дальше меняется только богатство семейства и масштаб данных.

Все, что мы будем изучать, от бустинга до GPT, укладывается в эти три предложения.

Доля это оценка

Монетка

63 орла из 100

p = 0.63

Слово

после «я» 120 раз из 1000 шло «хочу»

P(хочу | я) = 0.12

Вероятность следующего слова это доля. Своя монетка на каждое слово-предшественник, и сторон у нее столько, сколько слов.

Языковая модель из счетчиков

Таблица биграмм

```
я → хочу      0.12
я → не        0.09
я → думаю     0.07
я → буду      0.05
я → знаю      0.04
…
```

Генерация

я → хочу → сказать → что → …

Каждое следующее слово выбирается по долям из строки предыдущего. Получается связный на вид бред.

Пары не было

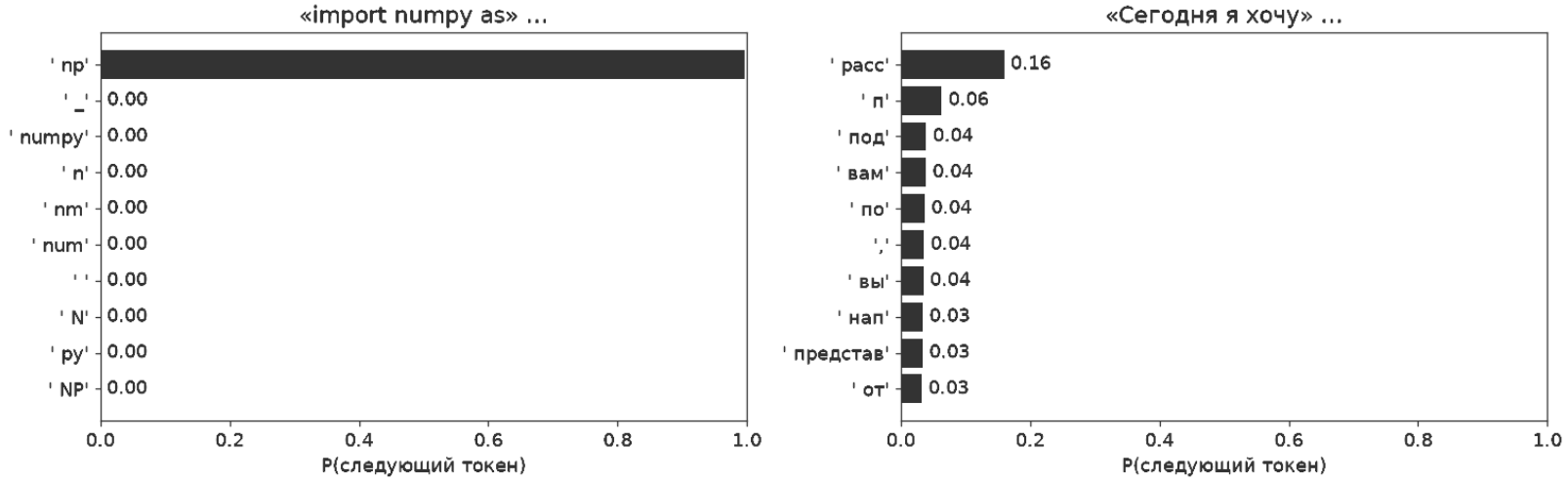
Вероятность ноль. Лечится сглаживанием.

Смысла нет

Модель помнит одно слово. Лечится длинным контекстом.

Работа 1: вы соберете такую модель без библиотек. Большая модель делает то же, только предшественников тысячи.

Что выдает настоящая модель



Qwen3-0.6B-Base, 600 миллионов параметров

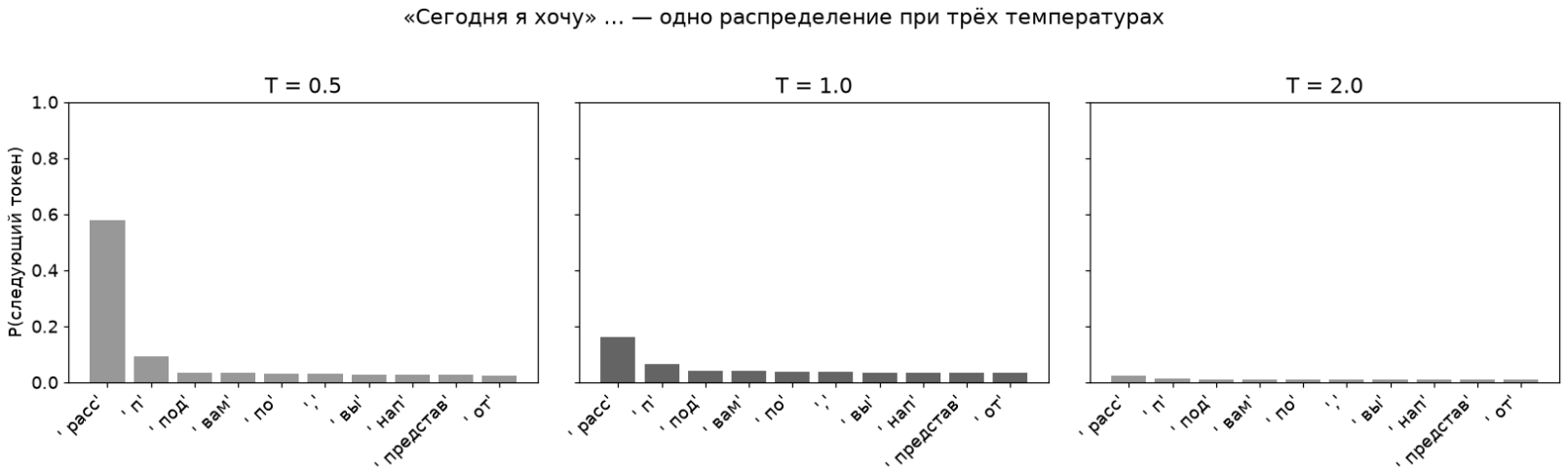
Слева: после «import numpy as» один столбик почти в единицу.

Справа: после «Сегодня я хочу» распределение размазано по сотням слов.

Та же таблица, что у биграмм, только предшественников тысячи и строка не хранится, а вычисляется.

На выходе модели не слово, а распределение по словарю. Слово из него выбирает код.

Температура



Одни и те же выходы модели

T = 0.5: распределение заостряется, почти всегда самое вероятное слово.

T = 1: как есть.

T = 2: сплющивается, в ответ попадает хвост редких слов.

Знаний не прибавилось и не убавилось, изменилась острота выбора. В работе 1 вы сделаете то же со счетчиками.

Температура делит логиты перед softmax. Та же формула будет у вас в работе 1: вероятности в степени 1/T.

Правдоподобие: монетка и языковая модель

МОНЕТКА

$$\log L = 63 \log p + 37 \log(1 - p)$$

Один параметр, сто бросков

=

ЯЗЫКОВАЯ МОДЕЛЬ

$$\log L = \sum_{t=1}^{T} \log P(x_t \mid x_{<t})$$

Миллиарды параметров, триллионы токенов

Обучение языковой модели это подбор миллиардов параметров, при которых тексты корпуса наиболее правдоподобны. Вероятности, логарифм, сумма, максимум. То же, что мы делали с монеткой.

Модель видит номера

ВРАЧ ОСМОТРЕЛ ПАЦИЕНТА

В	ра	ч	ос	мотр	ел	паци	ента

восемь токенов, каждый это номер в списке из 151 936

Слов бесконечно много

Формы, опечатки, имена, новые термины. Любой список слов завтра встретит слово, которого в нем нет.

У русского морфология

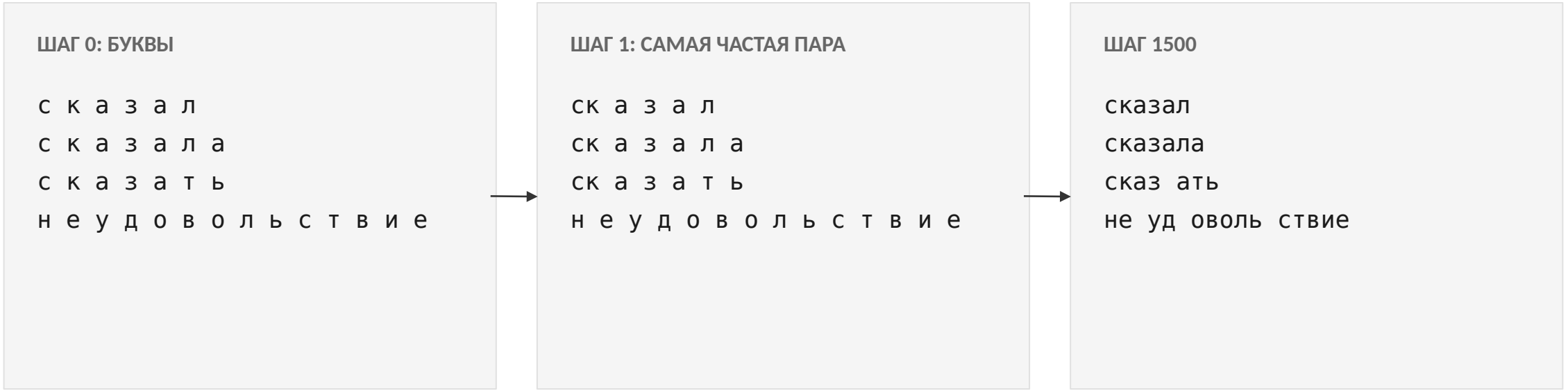
«Сказал, сказала, сказали, скажет, скажи»: пять записей в списке слов при одном смысле.

Байты слишком мелко

Страница текста это четыре тысячи шагов по одной букве. На смысл сил не остается.

Токен это компромисс: частое слово целиком, редкое кусками, незнакомое из букв.

Откуда берется список: BPE (Byte Pair Encoding)



Склеить самую частую пару соседних символов, повторить. Число склеиваний это размер словаря.

Словарь не придумывается, а формируется из букв по частоте.
У Qwen3 около 150 тысяч токенов, а у BPE в лабораторной 1500.

Стоимость в токенах

ОДНА И ТА ЖЕ ФРАЗА

В	ра	ч	ос	мотр	ел	паци	ента	и	сказал	,	что	его	ана	ли	зы	готов	ы	.	**19 токенов**	
The	doctor	examined	the	patient	and	said	that	his	tests	were	ready	.	**13 токенов**							

РУССКИЙ, АБЗАЦ В 100 СЛОВ

1.98 токена на слово

в контекст на 4096 токенов помещается около 2 тысяч слов

АНГЛИЙСКИЙ, АБЗАЦ В 70 СЛОВ

1.16 токена на слово

в контекст на 4096 токенов помещается около 3.5 тысяч слов

Контекст и цена измеряются в токенах, а не в словах. LLM-сервис обязан их считать.

Уверенная чушь

ПРОМПТ

Приведи пример использования torch.nn.functional.smooth_relu и объясни параметр beta.

ОТВЕТ МОДЕЛИ · Qwen3-0.6B, температура 0.7

Приведем пример использования torch.nn.functional.smooth_relu и объясню параметр beta.

Пример использования:

```
import torch
import torch.nn.functional as F

# Создаем входные данные
x = torch.randn(100, 100)  # 100x100 tensor

# Применяем функцию smooth_relu
output = F.smooth_relu(x, beta=0.1)

print(output)
```

Объяснение:

F.smooth_relu(x, beta=0.1) — это функция, которая применяет ReLU к входному тензору x, но с смягчением параметра beta, что уменьшает "ненужные" изменения по сравнению с обычным ReLU.

[…]

Функции torch.nn.functional.smooth_relu не существует. Сигнатура, параметр и пример придуманы.

СЛУЧАЙ

Спросил модель про функцию библиотеки, которой не существует. Получил сигнатуру, описание параметра и пример кода.

В ЧЕМ ОШИБКА

Факта нет. Функции нет. Все придумано, и выглядит безупречно.

МЕХАНИЗМ

Модель обучена максимизировать правдоподобие текста, а не истинность утверждений. Правдоподобный текст про несуществующую функцию это ровно то, что оптимизировалось. Уверенный тон не признак знания, это свойство вероятной последовательности токенов.

Главный вывод

Галлюцинация не баг реализации, а следствие целевой функции.

«Улучшением модели» она не чинится. Чинится снаружи: поиск по источникам, верификация, тесты.

Работа 1: языковая модель из счетчиков и свой токенизатор

Чистый Python без библиотек машинного обучения.

Языковая модель из счетчиков

Корпус «Война и мир», слова регулярным выражением, последние 10 % отложить

Вероятность следующего слова как доля, сглаживание

Генерация с температурой, правдоподобие связного и перемешанного

Свой токенизатор BPE

Слова в буквы с маркером конца слова

Склеить самую частую пару символов, повторить 1500 раз

Применить к корпусу, посчитать токенов на слово

Все, что было в этой паре, вы соберете сами: доля как оценка, температура, правдоподобие, перплексия. Во второй паре на этих же данных вырастет трансформер.

После перерыва

1. Как устроен трансформер: внимание и стопка слоев
2. Как его обучают: одна задача, одно число, которое уменьшают
3. Лабораторная работа 2: свой трансформер на тех же данных и на вашем токенизаторе

Принцип останется тем же. Изменится семейство: от таблицы счетчиков к нейронной сети.